



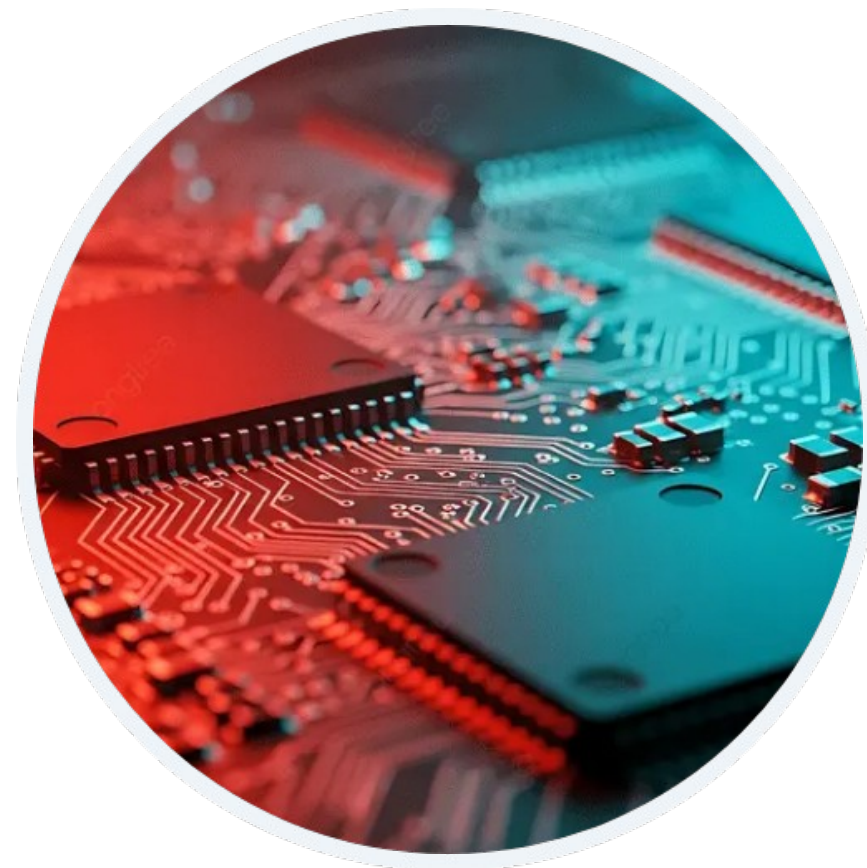
Newton School of AI & Computing

COMPUTER ORGANIZATION

UNIT 2: THE CENTRAL PROCESSING UNIT (CPU)

Presented By: Bibrata Sarkar

Department: Newton School of AI & Computing





[Dept Logo]

SYLLABUS OVERVIEW OF UNIT 2

Part A: CPU Internal Structure

- **Basic CPU Structure:** Core internal components.
- **Registers:** General and Special
- **Key Registers:** Program Counter (PC), Instruction Register (IR), Accumulator.
- **Instruction Cycle:** Fetch, Decode, Execute, Write Back phases.

Part B: Architecture & Data

- **Instruction Formats:** Zero, One, and Two Address.
- **Addressing Modes:** Immediate, Direct, Indirect, Register, etc.
- **Data Representation:** Signed, Unsigned, IEEE-754 Floating Point.
- **Control Unit Design:** Hardwired vs. Microprogrammed Control.

INDUSTRY APPLICATIONS

Where is CPU Architecture knowledge actively used in the tech industry?

- **Microprocessor Giants:** Intel (x86), AMD, and Apple Silicon (M-Series ARM architectures).
- **Mobile Computing:** Qualcomm Snapdragon and Mediatek processor design.
- **Servers & Cloud Computing:** Designing highly efficient data centers using AWS Graviton chips.
- **Chip Hardware:** Optimizing CPU control pathways for GPUs (NVIDIA) and NPUs.
- **Embedded Systems:** Creating logic for robotics, automotive control units (ECUs), and IoT devices.



COURSE OBJECTIVES

-
-
-
-
-



COURSE OUTCOMES (CO)

Upon successful completion of this course, students will be able to:

-
-
-
-
-



PROGRAM OUTCOMES (PO)

-
-
-
-
-
-



CO – PO MAPPING



PROGRAM SPECIFIC OUTCOMES (PSO)

-
-
-



CO – PSO MAPPING



PROGRAM EDUCATIONAL OBJECTIVES



RESULT ANALYSIS TEMPLATE



QUESTION PAPER TEMPLATE



PREREQUISITE

INTRODUCTION TO COMPUTER ORGANIZATION

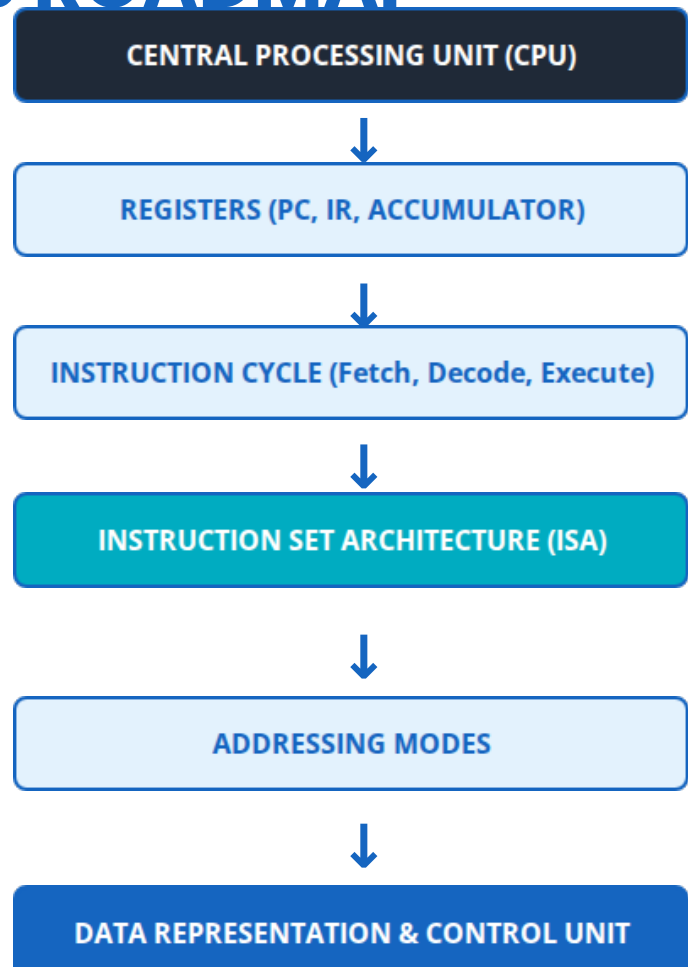
Why is the CPU called the Brain of the Computer?

- Just as the human brain coordinates all body parts, the CPU coordinates Memory, Input, and Output operations.
- It performs all the arithmetic and logical calculations required to run Operating Systems and Applications.
- **Relationship:** Memory supplies instructions; the CPU executes them; I/O provides the data and displays the results.

Educational Video: How a CPU works internally
https://www.youtube.com/watch?v=cNN_tTXABUA



UNIT 2 CONTENTS ROADMAP





UNIT 2 OBJECTIVES

Learning objectives and expected outcomes for Unit 2:

- Trace the complete data path of an instruction from memory fetch to execution.
- Understand the specific role of critical CPU registers like the Program Counter and Instruction Register.
- Differentiate between Zero, One, and Two address instruction formats.
- Apply various addressing modes to compute the effective memory address of an operand.
- Encode decimal numbers into IEEE-754 floating-point binary format.
- Compare hardwired and microprogrammed control unit designs.



TOPIC 1: BASIC CPU STRUCTURE

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To understand the fundamental internal architecture and core components of a Central Processing Unit.

Topic Outcome: Students will be able to draw the CPU block diagram, identify internal buses, and explain the distinct roles of the ALU and Control Unit.



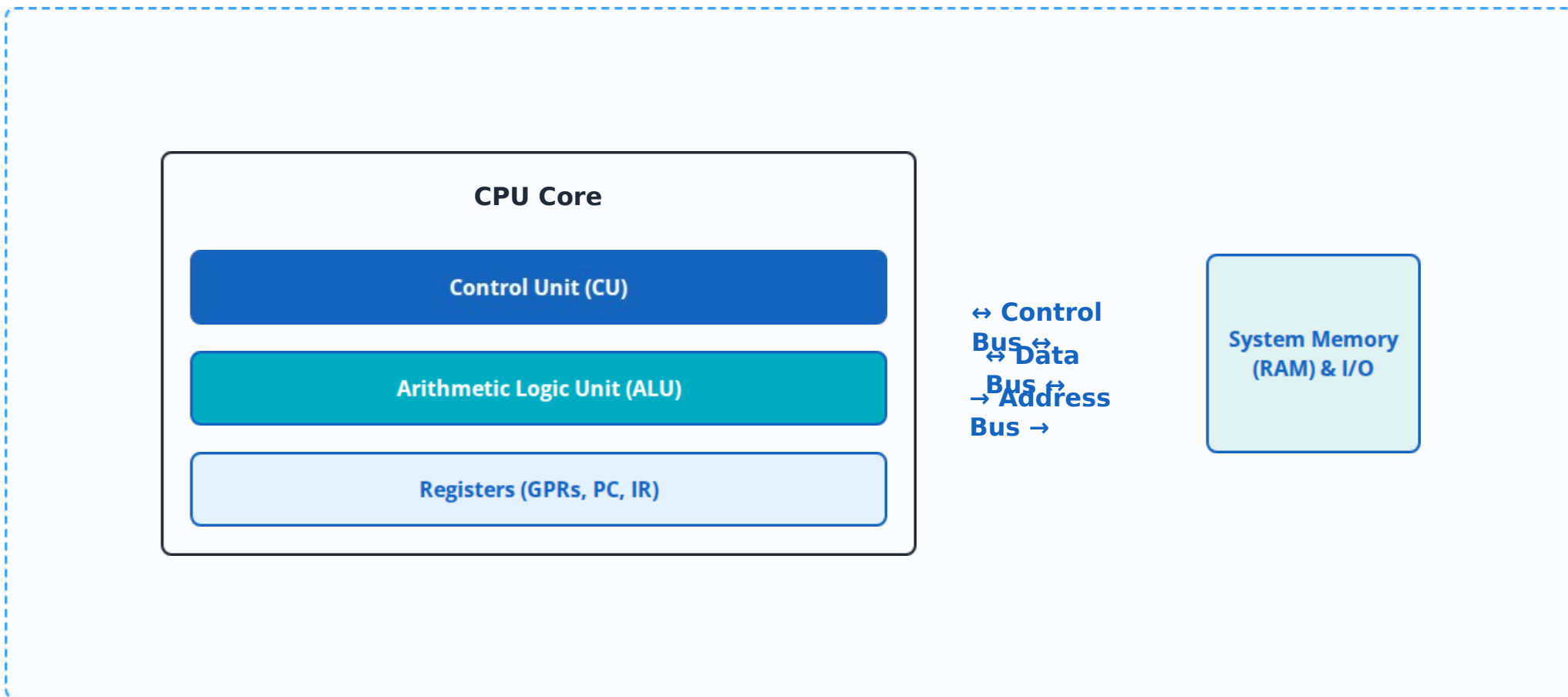
WHAT IS A CPU?

Definition: The Central Processing Unit (CPU) is the primary component of a computer that acts as its "brain." It performs basic arithmetic, logic, controlling, and input/output (I/O) operations specified by the instructions in the program.

CPU vs Processor vs Microprocessor:

- **Processor:** A general term for any logic circuitry that responds to and processes basic instructions.
- **CPU:** The primary processor in a computer system.
- **Microprocessor:** A CPU contained on a single integrated circuit (IC) chip. Modern CPUs are microprocessors.

CPU BLOCK DIAGRAM



FUNCTIONS OF THE CPU

Core Operations

- **Arithmetic:** Addition, Subtraction, Multiplication, Division.
- **Logical Operations:** AND, OR, NOT, comparisons (equal to, greater than).
- **Data Movement:** Moving data between memory and registers.

Management Operations

- **Control:** Generating timing signals to synchronize operations.
- **Instruction Execution:** Decoding software code into hardware actions.
- **Interrupt Handling:** Pausing tasks to respond to high-priority events (e.g., keyboard press).

WORKING OF CPU: PROCESS FLOW



Note: The CPU continually loops through steps 2, 3, and 4 millions of times per second (dictated by the Clock Speed, e.g., 3.0 GHz = 3 billion cycles/second).



CPU DAILY QUIZ

QUESTION 01

Which CPU component is responsible for performing arithmetic operations?

- A) Control Unit (CU)
- B) Arithmetic Logic Unit (ALU)
- C) Instruction Register (IR)
- D) System Bus

QUESTION 02

Where must data be located before the ALU can process it?

- A) Inside a Register
- B) Directly in RAM
- C) On the Hard Drive
- D) In the Cache exclusively

QUESTION 03

What is the correct order of the CPU processing loop?

- A) Decode → Fetch → Execute
- B) Fetch → Decode → Execute
- C) Execute → Fetch → Decode
- D) Fetch → Store → Decode

QUESTION 04

How does the CPU temporarily respond to high-priority external events?

- A) Clock Speed Acceleration
- B) Arithmetic Logic Operations
- C) Interrupt Handling
- D) Direct Data Movement to RAM



TOPIC 2: REGISTERS

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To examine the fastest memory components located directly inside the CPU.

Topic Outcome: Students will differentiate between General Purpose and Special Purpose registers and explain their absolute necessity for ALU operations.



INTRODUCTION TO REGISTERS

Definition: Registers are tiny, ultra-fast memory storage locations built directly into the CPU processor chip.

Purpose & Need: The ALU can ONLY process data if it is currently inside a register. It cannot do math directly on data sitting in RAM.

Why are they faster than RAM?

Registers are located on the exact same silicon die as the ALU, eliminating the physical distance and slow bus transfer times associated with reaching out to external RAM modules.

GENERAL PURPOSE REGISTERS (GPR)

Definition: Registers available to programmers and compilers to hold

intermediate data, operands, and memory pointers during calculations.

Examples: In x86 architecture, these are named EAX, EBX, ECX, EDX (or R0, R1, R2 in RISC architectures).

Real-life Analogy: A chef's cutting board.

You move vegetables (data) from the fridge (RAM) to the cutting board (GPR) to chop them quickly (ALU processing).

GPR

Bank

R1

R2

...

SPECIAL PURPOSE REGISTERS (SPR)

Definition: Registers reserved entirely for the Control Unit to manage the execution state of the CPU. Programmers cannot use them for general math.

Importance: They track where the program is, what it's doing, and the status of the last calculation.

Key Examples (Exam Important):

- **Program Counter (PC):** Points to the next instruction.
- **Instruction Register (IR):** Holds the current instruction.
- **Memory Address Register (MAR):** Holds the address being accessed in RAM.
- **Memory Data Register (MDR):** Holds the actual data read from/written to RAM.



REGISTER DAILY QUIZ

QUESTION 01

Which register holds the memory address of the next instruction to be fetched?

- A.** Instruction Register (IR)
- B.** Program Counter (PC)
- C.** Memory Address Register (MAR)
- D.** Memory Data Register (MDR)

QUESTION 02

What type of registers are EAX, EBX, ECX, and EDX in x86 architecture?

- A.** Special Purpose Registers
- B.** Control Registers
- C.** General Purpose Registers
- D.** Status Registers

QUESTION 03

Which register stores the actual data being read from or written to the RAM?

- A.** Memory Data Register (MDR)
- B.** Memory Address Register (MAR)
- C.** Instruction Register (IR)
- D.** Program Counter (PC)

QUESTION 04

Who primarily uses General Purpose Registers (GPRs) during CPU calculations?

- A.** Only the Control Unit
- B.** Programmers and Compilers
- C.** Only RAM and Cache controllers
- D.** The Input/Output Interface



TOPIC 3: PROGRAM COUNTER (PC)

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To understand the mechanism that sequences program execution.

Topic Outcome: Students will be able to trace how the Program Counter automatically increments to execute sequential code and how it handles jump/branch commands.

PROGRAM COUNTER WORKING

Definition: A special purpose register that holds the memory address of the *NEXT* instruction to be fetched from memory.

Step-by-Step Operation:

1. The CPU looks at the PC to find where in memory the next instruction is located.
2. The address is sent to the MAR.
3. As soon as the instruction is fetched, the PC automatically increments (e.g., $PC = PC + 1$) to point to the following instruction.

Real-Life Example: A bookmark in a book. It always points to the page you need to read next, moving forward as you finish reading the current page.



TOPIC 3: PROGRAM COUNTER (PC)

DAILY QUIZ: TEST YOUR UNDERSTANDING

QUESTION 1

What is the primary function of the Program Counter (PC)?

- A) To decode the fetched binary instruction
- **B) To hold the memory address of the next instruction**
- C) To perform arithmetic and logical operations
- D) To store temporary variables during execution

QUESTION 2

Which analogy best describes how a Program Counter works?

- A) A calculator keeping track of a running sum
- B) A magnifying glass focusing on specific code details
- **C) A bookmark pointing to the next page to read**
- D) A shelf organizing folders in alphabetical order

QUESTION 3

When does the Program Counter automatically increment?

- **A) As soon as the current instruction is fetched**
- B) Only after the instruction has been fully decoded
- C) Right before the system reboots or powers off
- D) When an arithmetic overflow occurs in the ALU

QUESTION 4

How does the PC handle jump or branch commands?

- A) It stops sequencing and alerts the user
- B) It resets its memory address register to zero
- **C) It loads the branch target address as the next instruction**
- D) It delegates the task directly to the ALU accumulator



TOPIC 4: INSTRUCTION REGISTER (IR)

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To identify where the CPU holds instructions during decoding.

Topic Outcome: Students will understand how the IR receives binary instructions from memory and feeds them directly to the Control Unit's decoder circuits.

INSTRUCTION REGISTER WORKING

Definition: A register that holds the current instruction that has just been fetched from memory and is about to be executed.



Execution Process: Once in the IR, the Control Unit strips apart the instruction to figure out the "Opcode" (What to do, e.g., ADD) and the "Operands" (What to do it to, e.g., R1, R2).



TOPIC 5: ACCUMULATOR

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To understand the primary operational register used in single-address processors.

Topic Outcome: Students will be able to explain how the Accumulator acts as both an implicit source and destination for ALU operations.



ACCUMULATOR (AC)

Definition: A highly specialized register used extensively by the ALU to store intermediate arithmetic and logical results.

Working (Implicit Role): In many older or simpler architectures, if an instruction says "ADD 5", it implicitly means "Add 5 to whatever is currently in the Accumulator, and store

the result back in the Accumulator."

Example Equation:

$AC = AC + M[\text{address}]$

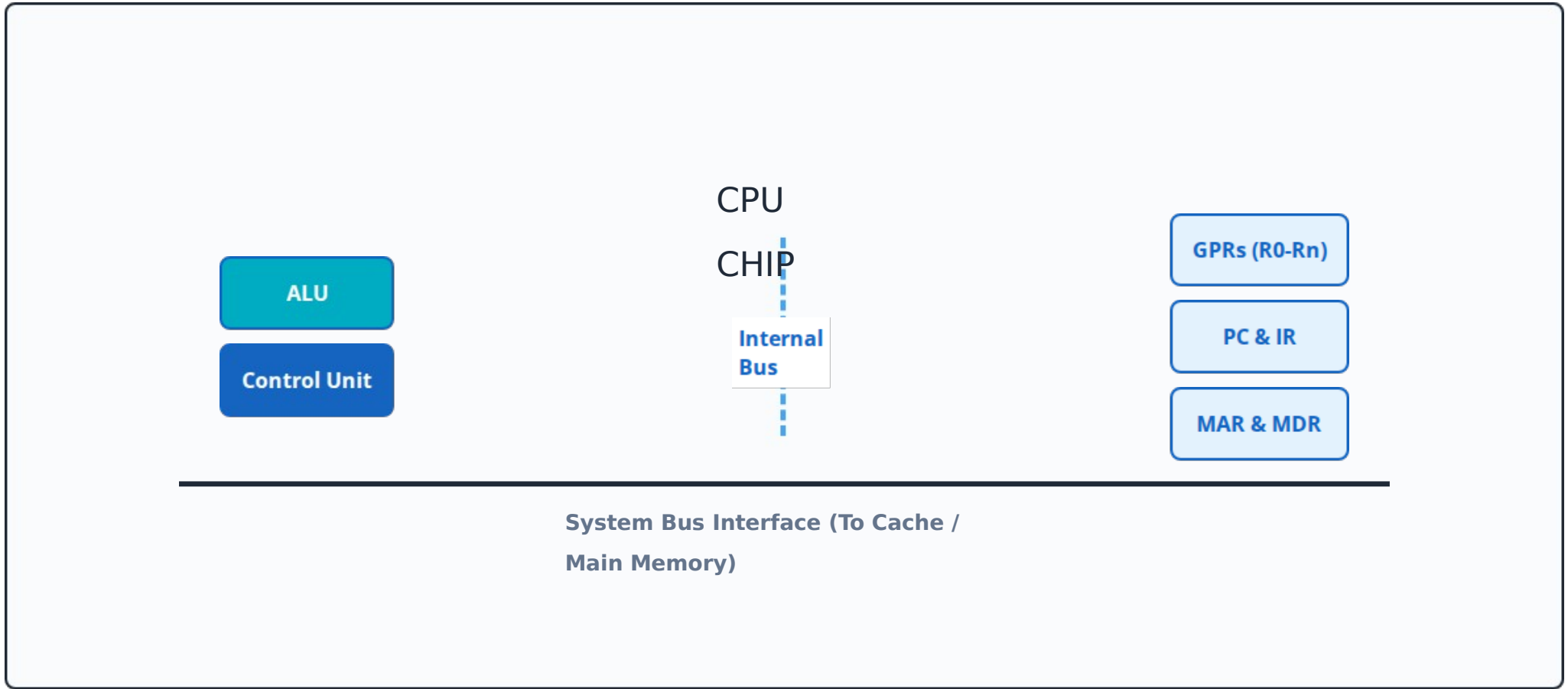
The Accumulator acts as a running total (like the display on a basic desktop calculator).

REGISTER COMPARISON

Register	Type	Primary Function	Programmer Access
GPR (R0, R1...)	General	Stores temporary data and operands during math.	Yes (Read/Write)
Program Counter (PC)	Special	Holds address of the NEXT instruction to fetch.	No (Directly)
Instruction Register (IR)	Special	Holds the CURRENT instruction being decoded.	No
Accumulator (AC)	Special/Gen	Holds implicit source and destination for ALU results.	Yes



CPU INTERNAL ORGANIZATION



DAILY QUIZ

DAILY QUIZ

Which register always points to the memory address of the very next instruction to be fetched?

A. Accumulator

B. Program Counter

C. Instruction Register

D. MAR

DAILY QUIZ

Where does the ALU store its intermediate mathematical results in a single-address architecture?

A. Accumulator

B. Program Counter

C. RAM directly

D. Control Unit

WEEKLY ASSIGNMENT & LINK



WEEKLY ASSIGNMENT 1

- 1 Draw the internal block diagram of a CPU showing ALU, CU, and registers connected via the internal bus.
- 2 Differentiate between the exact roles of the PC and IR.
- 3 Why are registers significantly faster than main memory (RAM)? (Provide architectural reasoning).

TOPIC LEARNING LINK

Platform: Youtube

Topic: CPU Architecture & Registers

<https://youtu.be/GtVDTp826DE?si=twlL6qjoIMxAM-54>



TOPIC 6: THE INSTRUCTION CYCLE

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To understand the continuous, repetitive sequence of events the CPU uses to process programs.

Topic Outcome: Students will be able to define the Instruction Cycle and list its basic phases (Fetch, Decode, Execute, Write Back) in strict chronological order.



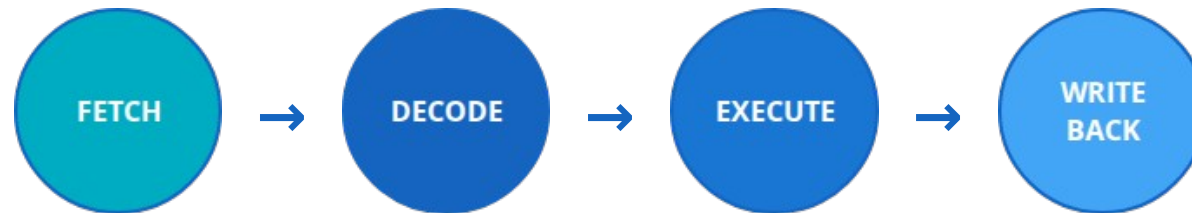
INTRO TO THE INSTRUCTION CYCLE

Definition: The time period during which one single instruction is fetched from memory and executed by the CPU. Every single line of software code written is ultimately broken down into machine instructions that pass through this identical multi-stage lifecycle.

Classroom Analogy (The Chef):

1. **Fetch:** Chef reads the next recipe card.
2. **Decode:** Chef understands "Ah, I need to chop onions."
3. **Execute:** Chef actually chops the onions.
4. **Write Back:** Chef places chopped onions into the storage bowl.

COMPLETE INSTRUCTION CYCLE



(Process repeats infinitely until the program halts)



TOPIC 7: FETCH CYCLE

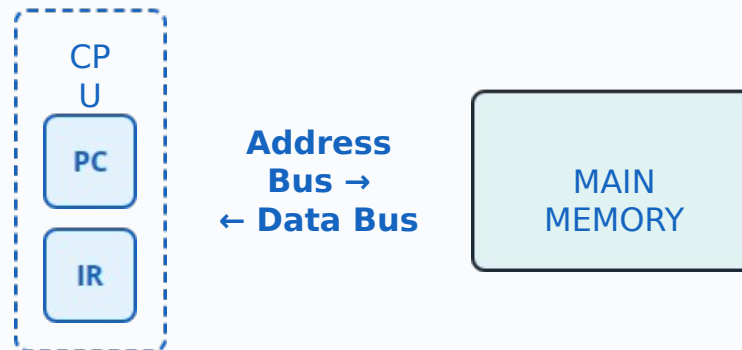
STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To examine the first phase of the instruction lifecycle.

Topic Outcome: Students will be able to trace how an instruction is physically moved from Main Memory into the CPU's Instruction Register using the PC, MAR, and MDR.

FETCH CYCLE DEFINITION

Definition: The process of retrieving an instruction from Memory (RAM) and placing it into the CPU.



FETCH CYCLE WORKING (STEP-BY-STEP)

1. $MAR \leftarrow PC$ (Address in PC is copied to Memory Address Register)
2. Control Unit issues READ command to Memory
3. $MDR \leftarrow M[MAR]$ (Memory places the instruction onto Data Bus, captured by MDR)
4. $IR \leftarrow MDR$ (Instruction is moved into the Instruction Register)
5. $PC \leftarrow PC + 1$ (Program Counter increments to point to the next instruction)



TOPIC 8: DECODE CYCLE

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To understand how the CPU interprets binary instructions.

Topic Outcome: Students will recognize the role of the Control Unit in splitting the instruction into Opcode and Operands, preparing the ALU for execution.

DECODE CYCLE WORKING

Definition: The Control Unit analyzes the instruction residing in the IR to determine exactly what operation to perform.

- **Opcode Identification:** Extracts the operation code (e.g., 0010 might mean ADD).
- **Operand Identification:** Determines WHERE the data is (e.g., in Register R1, or out in Memory).
- **Control Signal Generation:** The Control Unit prepares the electronic pathways (setting multiplexers) to route the correct data to the ALU.

IR Contents: [OPCODE (What)] | [OPERAND (Where)]



TOPIC 9: EXECUTE CYCLE

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To study the phase where the actual computational work is performed.

Topic Outcome: Students will explain how the ALU processes data based on the control signals generated during the decode phase.

EXECUTE CYCLE WORKING

Execution Types:

- **Arithmetic/Logical:** The ALU performs calculations on data from registers.
- **Memory Operation:** Data is loaded from memory (LOAD) or saved to memory (STORE).
- **Branch Instruction:** The PC is updated to jump to a different part of the program (e.g., loops, if-statements).

Real-Life Example: If the instruction decoded was "ADD R1, R2", the ALU physically passes the electrical signals from R1 and R2 through an Adder circuit.



TOPIC 10: WRITE BACK STAGE

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To understand how results are preserved.

Topic Outcome: Students will recognize the necessity of saving ALU results back into CPU Registers or Main Memory, updating status flags.



WRITE BACK WORKING

Definition: The final stage of the instruction cycle where the output from the Execute stage is written to a destination.

- **Register Update:** Storing math results in a General Purpose Register or Accumulator.
- **Memory Update:** Storing data permanently back to RAM (e.g., saving a variable).
- **Flags Update:** Updating condition codes (e.g., setting a 'Zero Flag' if the ALU result was 0).

Once Write Back completes, the CPU immediately returns to FETCH for the next instruction.

COMPLETE EXAMPLE: ADD R1, R2

Instruction: ADD R1, R2 (Meaning: $R1 = R1 + R2$)

- **Fetch:** PC points to memory address 100. Instruction 'ADD R1, R2' is pulled into IR.
- **Decode:** Control unit reads IR. Recognizes Opcode is ADD. Connects R1 and R2 to ALU.
- **Execute:** ALU performs physical binary addition of values inside R1 and R2.
- **Write Back:** The sum generated by ALU is stored back into Register R1.

DAILY QUIZ (INSTRUCTION CYCLE)

DAILY QUIZ

During which specific cycle is the instruction split into Opcode and

A. Fetch Operand? B. Decode

C. Execute

D. Write Back

DAILY QUIZ

Which register is actively updated during the final step of the Fetch cycle?

A. ALU

B. Instruction Reg

C. Program Counter

D. Memory Data Reg

WEEKLY ASSIGNMENT & LINK



WEEKLY ASSIGNMENT 2

- 1 Draw a flowchart illustrating the exact
micro-operations required during a
Fetch Cycle.
- 2 Explain the importance of the Write
Back stage. What happens if results
aren't written back?
- 3 Detail the cycle stages for the
instruction 'LOAD R1, [Memory
Address]'.

TOPIC LEARNING LINK

Platform: Youtube

Topic: Instruction Cycle Details

[https://youtu.be/_W4Yd-yiyog?
si=XIRg3N6HdP1FBBW1](https://youtu.be/_W4Yd-yiyog?si=XIRg3N6HdP1FBBW1)



TOPIC 11: INSTRUCTION SET ARCHITECTURE

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To understand the critical interface boundary between software software programmers and hardware processors.

Topic Outcome: Students will be able to define ISA, list its core components, and explain how it dictates compiler behavior.



WHAT IS ISA?

Definition: Instruction Set Architecture (ISA) is the abstract model of a computer. It is the vocabulary that the CPU physically understands.

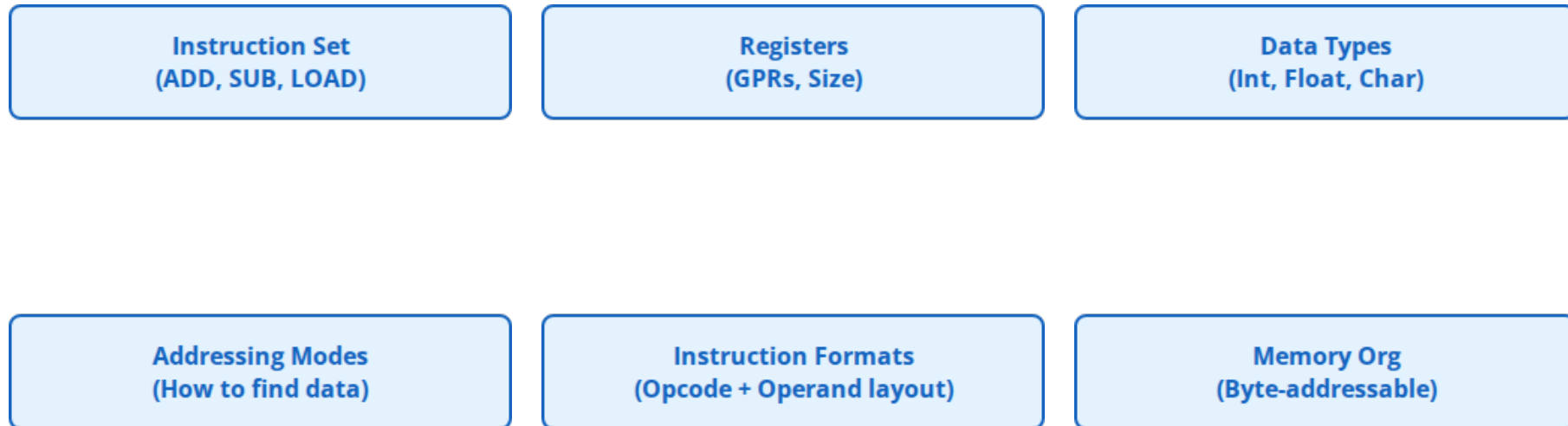
Need & Purpose: Software (like C++ or Python) cannot run directly on hardware. A compiler must translate software into the specific instructions (ISA) the CPU supports.

Difference from Computer Architecture:

ISA is the *programmer's view* of the machine (registers available, instructions supported). It does NOT specify how fast the CPU is, or how much cache it has (that is microarchitecture/organization).



COMPONENTS OF ISA



A complete ISA defines ALL of these parameters for a processor family (e.g., x86, ARM).



TOPIC 12: INSTRUCTION FORMATS

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To analyze how a binary instruction is physically structured into distinct fields.

Topic Outcome: Students will differentiate between Zero, One, and Two Address instructions, analyzing their impact on program length and CPU architecture.

INSTRUCTION FORMAT LAYOUT

An instruction is a sequence of bits divided into specific fields. Typical layout:



OPERAND 2 (Src)

- **Opcode:** Specifies the operation to be performed (e.g., ADD, SUB).
- **Operand:** Can be a direct memory address, a register reference, or raw data.
- **Instruction Length:** Dictated by CPU architecture (e.g., 16-bit, 32-bit).

ZERO ADDRESS INSTRUCTIONS

Definition: Instructions that contain NO explicit operand addresses. They only contain an

Architecture: Exclusively used in Stack-Organized computers.

Working: Operands are implicitly taken from the top of the Stack. Results are pushed back to the top of the stack.

Example: ADD (Pops top two items from stack, adds them, pushes result back).

- **Advantage:** Very short instructions (saves memory).
- **Disadvantage:** Complex stack management, slower execution for complex math.

ONE ADDRESS INSTRUCTIONS

Definition: Instructions containing an Opcode and exactly ONE explicit address.

Architecture: Exclusively used in Accumulator-Organized computers.

Working: The Accumulator (AC) is the implicit second operand AND the destination.

Example: ADD X (Means: $AC = AC + M[X]$)

- **Advantage:** Short instructions, simple hardware logic.
- **Disadvantage:** Accumulator becomes a bottleneck; requires many instructions to do complex equations (Load, Add, Store sequences).

TWO ADDRESS INSTRUCTIONS

Definition: Instructions containing an Opcode and TWO explicit addresses (commonly **Architecture Standard**) in General Purpose Register (GPR) architectures (like x86).

Working: Can specify multiple registers or memory locations directly.

Example: ADD R1, R2 (Means: $R1 = R1 + R2$)

- **Advantage:** Reduces total program length (fewer instructions needed). Flexible
- **Disadvantage:** Longer instruction format (requires more bits to encode two addresses).

COMPARISON OF INSTRUCTION FORMATS

Feature	Zero Address	One Address	Two Address
Implicit Component	Top of Stack	Accumulator (AC)	None (Explicitly defined)
Instruction Size	Very Small	Small	Large
Program Length	Very Long (Many lines)	Medium	Short (Fewer lines)
Example	ADD	ADD X	ADD R1, X
Primary Architecture	Stack Processors	Legacy Processors	Modern GPR Processors

DAILY QUIZ (ISA & FORMATS)

DAILY QUIZ

An architecture that strictly uses One Address instructions relies heavily on

A. which register?
Accumulator

B. Program Counter

C. GPR R1

D. Stack Pointer

DAILY QUIZ

In a Zero Address format, the ADD instruction implicitly grabs operands

A. Memory directly from: B. Top of the stack

stack



TOPIC 13: ADDRESSING MODES

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To explore the various techniques a CPU uses to locate and retrieve data required for an instruction.

Topic Outcome: Students will be able to define 'Effective Address' and compute it using Immediate, Direct, Indirect, and Register modes.



INTRODUCTION TO ADDRESSING MODES

Definition: The rules and methods used by a processor to specify the operand (data) for an instruction.

Effective Address (EA): The actual, physical memory address where the operand is

definitively stored.

Why do we need multiple modes?

- To give programmers versatility (accessing arrays, pointers, constants).
- To reduce the number of bits in the addressing field of the instruction.
- To increase instruction execution efficiency.

IMMEDIATE ADDRESSING MODE

Working: The operand is physically present *inside* the instruction itself. No memory access is required to fetch the data.

Instruction Example: ADD 5 (Add the constant number 5 to the accumulator).

OPCODE
(ADD)

OPERAND
(Data = 5)

- **Advantage:** Fastest mode. No extra memory fetch cycle needed.
- **Disadvantage:** The size of the number is limited by the size of the address field.

DIRECT ADDRESSING MODE

Working: The instruction contains the exact memory address where the operand is stored.

Instruction Example: LOAD 1000 (Load the data currently sitting at memory location

1000).

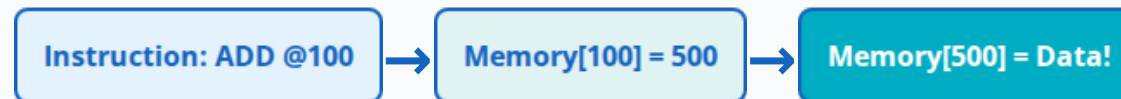
Formula: Effective Address (EA) = Address field value.

- **Advantage:** Simple logic.
- **Disadvantage:** Requires exactly 1 memory fetch to get the data. Address space is limited by instruction size.

INDIRECT ADDRESSING MODE

Working: The instruction contains a memory address, but that memory location holds *another* address, which finally points to the actual data. (Pointers).

Formula: Effective Address (EA) = Memory[Address field value].



- **Advantage:** Enormous address space flexibility (great for pointers/arrays).
- **Disadvantage:** Slow. Requires 2 memory fetches to get one piece of data.



REGISTER ADDRESSING MODE

Working: The operand is stored in a CPU Register, and the instruction specifies which

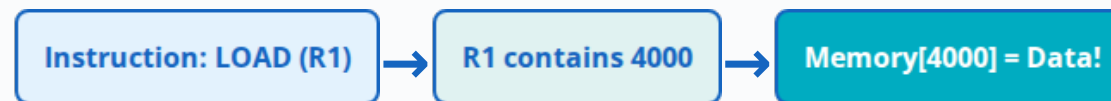
Instruction Example: ADD R1 (Add data in R1 to accumulator).

Formula: Effective Address (EA) = Register R1.

- **Advantage:** Extremely fast. No memory fetch is required since data is already in the register.
- **Disadvantage:** Extremely limited number of registers available.

REGISTER INDIRECT ADDRESSING MODE

Working: The instruction specifies a CPU Register. That register contains the memory address of the actual data.



Advantage: Faster than memory-indirect because getting the pointer from the register takes zero memory cycles. (Requires only 1 memory fetch to get the data).

COMPARISON OF ADDRESSING MODES

Addressing Mode	Effective Address (EA)	Memory Fetches	Advantage
Immediate	Operand is the data	0	Fastest execution
Register	EA = Register	0	Very fast, no RAM access
Direct	EA = Address Field	1	Simple logic
Register Indirect	EA = Value in Register	1	Excellent for pointers
Indirect	EA = Memory[Address]	2	Massive addressing range

DAILY QUIZ (ADDRESSING MODES)

DAILY QUIZ

Which addressing mode requires ZERO memory fetches to retrieve the

A. Immediate operand? B. Direct

C. Indirect

D. Index

DAILY QUIZ

In which mode does the instruction provide a pointer to a pointer?

A. Direct

B. Register

C. Indirect

WEEKLY ASSIGNMENT & LINK



WEEKLY ASSIGNMENT 3

- 1 Define Effective Address (EA).
 - Calculate the EA for Direct,
- 2 Indirect, and Immediate modes
Compare and contrast Zero
· given instruction 'LOAD 500'.
Address and Two Address
- 3 instruction formats with block
· Why are pointers in C
diagrams.
programming heavily reliant on
Register Indirect addressing?

TOPIC LEARNING LINK

Platform: Gate Smashers

Topic: Addressing Modes Explained

https://youtu.be/_CH4cm5PhK8?si=thFt7LRVTsCILObT



TOPIC 14: DATA REPRESENTATION

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To understand how negative values and decimals are physically stored in binary.

Topic Outcome: Students will differentiate between signed/unsigned integers and utilize IEEE-754 for floating point storage.



INTRODUCTION TO DATA REPRESENTATION

Computers only understand 0s and 1s. But real-world mathematical data includes:

- Positive integers (e.g., 50)
- Negative integers (e.g., -50)
- Real/Decimal numbers (e.g., 3.14159)

The Problem: A computer register doesn't have a "+" or "-" or "." symbol. It ONLY has switches. We must encode signs and decimal points entirely using 0s and 1s.



SIGNED VS UNSIGNED NUMBERS

Unsigned Numbers:

All bits in the register are used strictly for magnitude (value). Cannot represent negative numbers.
Example (8-bit): Range is 0 to 255.

Signed Numbers:

The Most Significant Bit (MSB - left most) is reserved as the **Sign Bit**.

- **0** = Positive Number
- **1** = Negative Number

Example (8-bit): 1 bit for sign, 7 bits for magnitude. Range is -128 to +127.

SIGNED VS UNSIGNED COMPARISON

Feature	Unsigned Numbers	Signed Numbers (2's Complement)
Sign Bit Reserved?	No	Yes (MSB is the sign bit)
8-bit Range	0 to 255	-128 to +127
16-bit Range	0 to 65,535	-32,768 to +32,767
Application	Memory Addresses, Counter variables	Mathematical equations, Banking apps



TOPIC 16: IEEE 754 FLOATING POINT

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To learn the global engineering standard for storing decimal/fractional numbers in hardware.

Topic Outcome: Students will encode decimal floating-point numbers into 32-bit single precision IEEE 754 format.



INTRO TO FLOATING POINT

Floating Point Numbers: Numbers with fractional parts (e.g., 3.14, -0.005, 1.0×10^9).

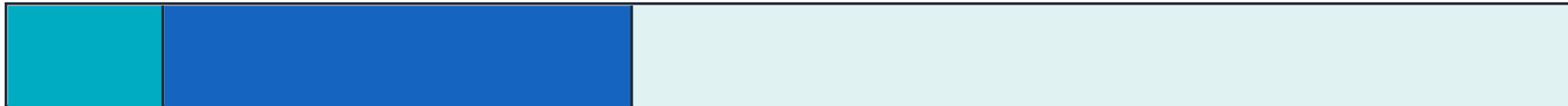
The Need: Fixed point representation (e.g., dedicating exact 16 bits for whole, 16 bits for fraction) is terribly inefficient for extremely large or extremely small numbers.

Scientific Notation Principle: Representing a number as **$M \times B^E$**
(Mantissa x Base ^ Exponent). This allows the decimal point to "float" based on the exponent.

Used heavily in 3D graphics (GPUs), scientific computing, and physics simulations.

IEEE 754 SINGLE PRECISION FORMAT

A 32-bit standard format divided exactly into three distinct fields:



MANTISSA / FRACTION (23 bits)

- **Sign (S):** 0 for positive, 1 for negative.
- **Exponent:** Shifted by a bias of 127 ($\text{Value} = \text{True Exponent} + 127$).
- **Mantissa:** The normalized fractional part. The leading '1' is implicit (hidden) to save a bit.

IEEE 754 WORKING EXAMPLE

Example: Convert -9.75 to IEEE 754 Single Precision.

1. **Sign Bit:** Number is negative $\rightarrow S = 1$
2. **Binary Convert:** $9 = 1001$, $0.75 = .11 \rightarrow$ Binary is 1001.11
3. **Normalize:** $1.00111 \times 2^3 \rightarrow$ (True Exponent = 3)
4. **Biased Exponent:** $3 + 127 = 130 \rightarrow$ Binary 130 = **10000010**
5. **Mantissa:** Drop the leading 1. Remaining is 00111. Pad with 0s to 23 bits $\rightarrow$
00111000000000000000000

Final 32-bit: 1 | 10000010 | 00111000000000000000000

DAILY QUIZ (DATA REP)

DAILY QUIZ

In an 8-bit Signed (2's complement) integer representation, what is the maximum positive value?

A. +127

B. +128

C. +255

DAILY QUIZ

In IEEE-754 Single Precision, what is the exact bias added to the true

A. 128 exponent? B. 127

C. 1023



TOPIC 17: HARDWIRED CONTROL

STEP 1: TOPIC OBJECTIVE & OUTCOME

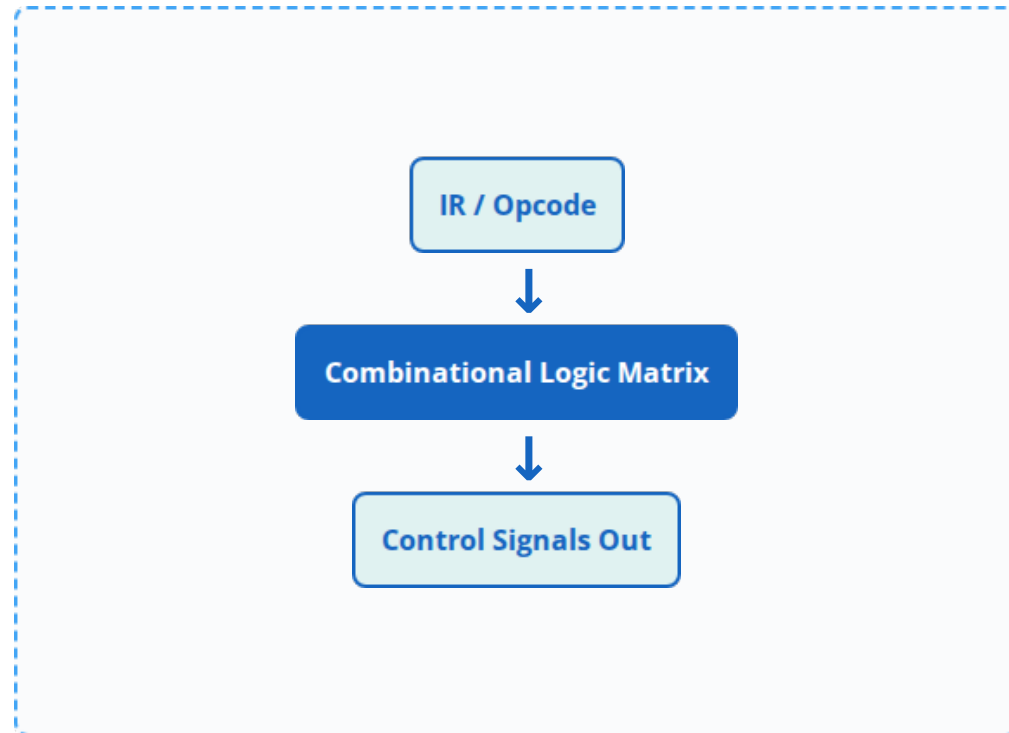
Topic Objective: To investigate how the Control Unit generates signals using pure hardware circuits.

Topic Outcome: Students will analyze the physical architecture of a Hardwired CU and its speed advantages in RISC processors.

HARDWIRED CONTROL UNIT

Definition: A Control Unit design where control signals are generated purely by interconnected physical logic gates, flip-flops, and decoders.

- **Inputs:** Instruction Register (Opcode), Timing signals (Clock), Condition Flags.
- **Advantages:** Extremely FAST execution. (Zero memory lookup required).
- **Disadvantages:** Inflexible. To add a new instruction, you must physically resolder/redesign the silicon chip. Very complex for large instruction sets.





TOPIC 18: MICROPROGRAMMED CONTROL

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To explore software-driven control unit designs.

Topic Outcome: Students will differentiate between hardwired logic and micro-instructions stored in a Control Memory.



MICROPROGRAMMED CONTROL UNIT

Definition: A Control Unit design where control signals are generated by reading "micro-instructions" from a special fast internal ROM (Control Memory). Instead of hard physical wires, the steps to execute an instruction are written as a tiny program (Microprogram) stored inside the CPU itself.

- **Advantages:** Extremely flexible. Easy to update instructions or add new ones by simply flashing the ROM. Reduces hardware complexity for massive CISC processors.

- **Disadvantages:** Slower. Generating a signal requires a memory fetch from the Control ROOM.

HARDWIRED VS MICROPROGRAMMED

Feature	Hardwired Control Unit	Microprogrammed Control Unit
Implementation	Physical Logic Gates & Flip-Flops	Micro-instructions in ROM
Speed	Very Fast	Slower (Requires memory access)
Flexibility	Rigid (Requires hardware redesign)	Highly Flexible (Software updatable)
Cost	Expensive for complex sets	Cheaper for large instruction sets
Architecture Used	RISC Processors (ARM, MIPS)	CISC Processors (Intel x86)

COMPLETE UNIT MCQS

- Q1.** Which register holds the instruction currently being decoded? -> **(B) IR**
- Q2.** What is the formula for Effective Address in Direct Addressing? -> **(A) EA = Address Field**
- Q3.** Zero Flag instructions are strictly used in which architecture? -> **(C) Stack-Organized**
- Q4.** In IEEE 754, what does a Sign bit of '1' signify? -> **(D) Negative Number**
- Q5.** Which Control Unit design is easiest to modify? -> **(A) Microprogrammed**



GLOSSARY

- **CPU:** Central Processing Unit (ALU + CU).
- **GPR:** General Purpose Register for math.
- **Program Counter (PC):** Points to NEXT instruction.
- **Accumulator:** Implicit ALU destination register.
- **Instruction Cycle:** Fetch, Decode, Execute, Write.
- **ISA:** Programmer's hardware vocabulary.
- **Addressing Mode:** Method to find effective address.
- **IEEE 754:** Floating-point standard (Sign, Exp, Mantissa).
- **Hardwired:** Gate-based fast control
- **Microprogrammed:** ROM-based flexible control unit.



PRACTICE QUESTIONS

Short Questions (2 Marks):

1. Define Effective Address (EA).
2. Differentiate between PC and IR.
3. State the bias value for IEEE 754 Single Precision.

Long Questions (10 Marks):

1. With a neat diagram, explain the complete Instruction Cycle.
2. Compare Hardwired and Microprogrammed Control Units.
3. Explain Immediate, Direct, and Register Indirect addressing modes with examples.



EXPECTED UNIVERSITY QUESTIONS

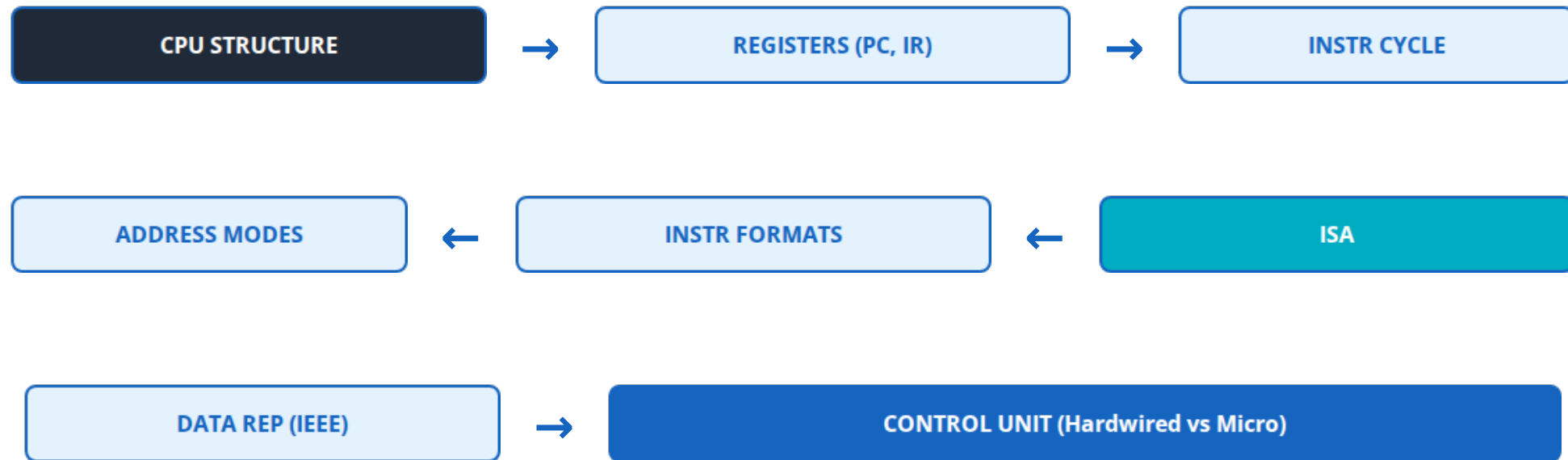
High-Frequency Numerical Questions:

- Convert the decimal number -12.5 into IEEE 754 Single Precision Floating Point format.
- (Highly Expected) Given an instruction LOAD [800] and Register R1=400, calculate EA for Direct and Register Indirect modes.

Most Expected Diagram Questions:

- Draw the detailed block diagram of CPU internal organization showing internal buses.
- Draw the block diagram illustrating Hardwired Control logic.

UNIT 2 RECAP



(1-Minute Revision: CPU fetches via PC->IR, decodes based on ISA/Modes, executes in ALU, controlled by Hardwired/Micro logic).

THANK YOU

COMPUTER ORGANIZATION

UNIT 2 – THE CENTRAL PROCESSING UNIT

Questions & Discussion

